

Emotion Detection and Learning Support Engine

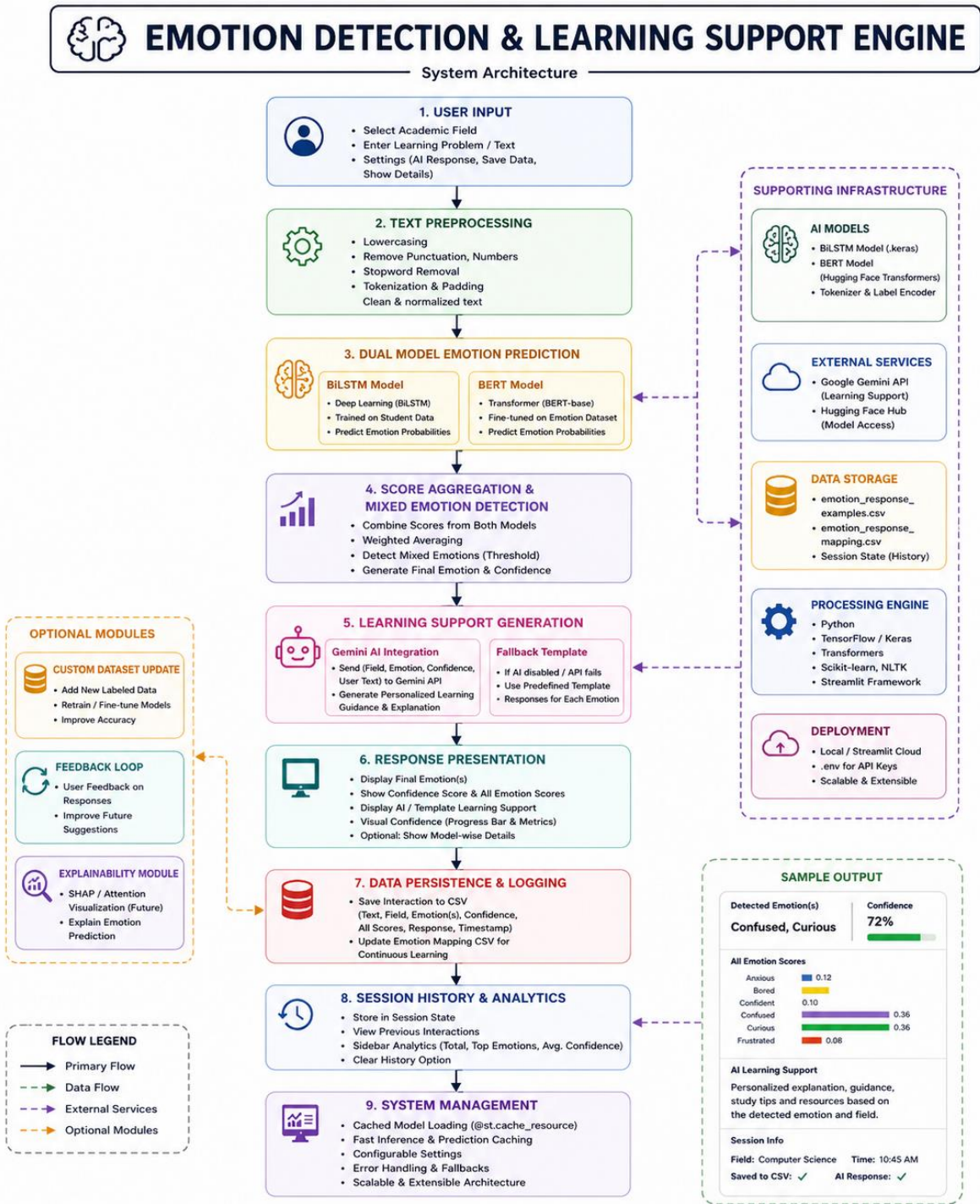
Project Description:

Emotion Detection and Learning Support Engine is an AI-powered educational assistance system designed to understand students' emotional states from their learning-related text and provide personalized academic support. The application combines a deep learning **BiLSTM** model and a **BERT-based Transformer** model to accurately classify student emotions such as **anxious, bored, confident, confused, curious, and frustrated**. A confidence-based prediction mechanism, along with mixed-emotion analysis, enables the system to capture complex emotional expressions rather than relying solely on a single-label prediction. The project is developed using **Python, TensorFlow, Hugging Face Transformers, Scikit-learn, and NLTK**, while the interactive user interface is built with **Streamlit**. The system also incorporates optimized model loading, prediction caching, and detailed confidence visualization to ensure a smooth and responsive user experience.

Beyond emotion detection, the project functions as an intelligent learning support platform by integrating **Google Gemini AI** to generate personalized learning guidance based on the detected emotion and the student's academic field. The application provides contextual explanations, motivational feedback, and actionable study recommendations to help students overcome learning challenges and improve engagement. It further maintains session history, stores interactions in CSV files for continuous learning and future analysis, and offers configurable settings for AI-generated responses, prediction details, and data persistence. By combining natural language processing, deep learning, generative AI, and an intuitive web interface, the Emotion Detection and Learning Support Engine demonstrates the practical application of Artificial Intelligence in enhancing personalized education, emotional awareness, and adaptive learning experiences.

Technical Architecture:

Technical Architecture:



Description:

The Emotion Detection and Learning Support Engine is an AI-powered educational assistance system designed to identify students' emotional states from their learning-related text and provide personalized academic guidance. The system combines BiLSTM and BERT deep learning models to accurately detect emotions such as Anxious, Bored, Confident, Confused, Curious, and Frustrated. It follows a structured processing pipeline that begins with user input, performs text preprocessing, predicts emotions using dual AI models, generates AI-powered learning support, and stores interaction data for continuous learning. The application is developed using Python, TensorFlow, Hugging Face Transformers, Scikit-learn, NLTK, and Streamlit, while Google Gemini AI is integrated to generate intelligent, personalized learning recommendations based on the detected emotion and the student's academic field.

1. User Input

The process begins when the user selects an academic field and enters a learning-related question, doubt, or problem through the Streamlit web interface. Users can also configure system settings such as enabling AI-generated responses, saving interaction history, and displaying detailed prediction results.

Purpose:

Accept student learning-related text.

Select the relevant academic field.

Configure system options before analysis.

2. Text Preprocessing

The entered text is cleaned and normalized before it is passed to the prediction models. This stage improves the quality of the input and ensures consistent model performance.

Operations:

Convert text to lowercase

Remove punctuation and special characters

Stop-word removal

Tokenization

Sequence padding

Text normalization

Purpose:

Clean noisy input text.

Improve prediction accuracy.

Prepare text for deep learning models.

3. Dual Model Emotion Prediction

The preprocessed text is analyzed simultaneously using two deep learning models:

BiLSTM Model – Captures sequential contextual information from text.

BERT Model – Utilizes transformer-based contextual embeddings for deeper language understanding.

Both models generate emotion probabilities for each predefined emotion category.

Detected Emotions:

Bored

Confident

Confused

Curious

Frustrated

Purpose:

Predict student emotions accurately.

Utilize complementary strengths of BiLSTM and BERT.

Improve robustness through dual-model prediction.

4. Score Aggregation & Mixed Emotion Detection

The prediction scores from both models are combined using a weighted aggregation strategy. The system also detects multiple significant emotions when confidence values exceed a predefined threshold.

Operations:

Combine BiLSTM and BERT scores

Weighted confidence calculation

Mixed emotion detection

Final emotion selection

Purpose:

Increase prediction reliability.

Detect multiple emotional states.

Generate final emotion and confidence score.

5. AI Learning Support Generation

Based on the detected emotion, confidence score, academic field, and user input, the system generates personalized learning assistance using Google Gemini AI. If AI services are unavailable, predefined template-based responses are provided.

Functions:

Personalized learning guidance

Emotional encouragement

Study recommendations

Concept explanations

Fallback response generation

Purpose:

Provide adaptive educational support.

Help students overcome learning difficulties.

Deliver context-aware AI-generated responses.

6. Response Presentation

The generated learning support and emotion prediction results are presented through an interactive Streamlit dashboard.

Displayed Information:

Final detected emotion(s)

Confidence score

Emotion probability distribution

Personalized AI response

Optional detailed model predictions

Purpose:

Present predictions clearly.

Visualize model confidence.

Improve user understanding of the analysis.

7. Data Persistence & Logging

Each interaction can be stored for future analysis and continuous learning. The system maintains structured logs and response datasets in CSV format.

Stored Information:

Student text

Academic field

Predicted emotion

Confidence score

AI response

Timestamp

Emotion scores

Purpose:

Maintain historical records.

Support future model improvement.

Enable continuous learning and analytics.

8. Session HiActivity & Analytics

The application maintains the user's interaction hiActivity during the session and provides useful analytics regarding previous predictions.

Features:

Session hiActivity

Previous interactions

Emotion statistics

Confidence summaries

Clear hiActivity option

Purpose:

Track learning progress.

Review previous analyses.

Improve user experience.

9. Streamlit Dashboard

The final system is deployed as an interactive Streamlit web application, providing a simple and responsive interface for students and educators.

Dashboard Features:

Select academic field

Enter learning problem

Generate emotion prediction

Display AI learning support

Supporting Infrastructure

The **Emotion Detection and Learning Support Engine** incorporates several supporting components that ensure accurate emotion prediction, efficient processing, intelligent response generation, and a seamless user experience.

AI Models

The system utilizes multiple Artificial Intelligence models for emotion detection and personalized learning support.

Pre-trained Models Include:

- **BiLSTM Emotion Classification Model** (TensorFlow/Keras)
- **BERT Emotion Classification Model** (Hugging Face Transformers)
- **Google Gemini AI** (Personalized Learning Support)
- **Tokenizer & Label Encoder**
- **Student Adaptive Fine-tuned Model (Optional)**

Data Storage

The application stores interaction data and prediction results for continuous learning and future analysis.

Stored Data Includes:

- Student Learning Problems / Text
- Predicted Emotion
- Emotion Confidence Scores
- Emotion Probability Distribution
- AI Learning Responses
- Academic Field Information
- Session HiActivity
- CSV Logs (Interaction HiActivity)

- Response Mapping Dataset

Processing Engine

The core processing engine is built using modern AI and Machine Learning technologies.

Core Technologies Include:

- Python
- TensorFlow / Keras
- Hugging Face Transformers
- Scikit-learn
- NumPy
- Pandas
- NLTK
- Google GenAI SDK
- Streamlit Framework

Visualization Tools

Visualization components help present prediction results and model confidence in an intuitive manner.

Visualization Libraries:

- Streamlit Native Charts
- Plotly
- Matplotlib
- Progress Bars & Interactive Metrics

Deployment

The application is designed for flexible deployment across multiple environments.

Deployment Options:

- Local Machine
- Streamlit Cloud
- Python Virtual Environment (venv)
- Docker (Optional)
- Cloud-based Deployment (Future Enhancement)

Optional Modules

The architecture supports several optional modules that can be integrated to enhance system functionality and future scalability.

Learning Analytics Module

- Track user interaction history
- Analyze frequently detected emotions
- Generate learning progress summaries
- Provide performance analytics

Personalized Recommendation Module

- Adaptive study recommendations
- Emotion-aware learning guidance
- Field-specific educational resources
- AI-generated motivational feedback

Multi-language Support (Future Enhancement)

- Automatic language detection
- Multilingual emotion detection
- AI responses in multiple languages

- Regional language support

Model Management Module

- Cached model loading for faster inference
- Dynamic model switching (BiLSTM/BERT)
- Confidence threshold configuration
- Automatic fallback response handling

This supporting infrastructure closely matches the architecture and implementation of your **Emotion Detection and Learning Support Engine** project and aligns with the features you've implemented, including the dual-model prediction pipeline, Gemini AI integration, CSV logging, session hiActivity, analytics, and Streamlit-based deployment.

Pre-requisites:

Python Programming Proficiency:[Python Documentation](#)

Streamlit framework knowledge:[Streamlit Documentation](#)

Deep Learning (NLP/ASR) Familiarity: [Hugging Face Transformers Documentation](#)

Gemini API key:[ai studio](#)

Optional Tools for Development:VS code, Py charm

Model assets and data

Project Workflow:

Milestone 1: Environment Setup and Dependency Configuration

Activity 1.1: Obtain Gemini API Key

Activity 1.2: Install Python & Create Virtual Environment

Activity 1.3: Install Project Dependencies

Activity 1.4: Create the .Env Configuration File

Activity 1.5: Verify Model and Data Directories

Activity 1.6: Prepare Project Folder Structure

Milestone 2. Kaggle Model Training and Integration

Activity 1.1: Kaggle Setup (GPU + Dependencies + Data Loading)

Activity 1.2: Data Preprocessing & Tokenization

Activity 1.3: BiLSTM Model Training

Activity 1.4: Domain-Adaptive Fine-Tuning

Activity 1.5: BERT Model Fine-Tuning

Activity 1.6: Model Export & Local Integration

Milestone 3. Core Emotion Detection Pipeline Development

Activity 1.1: Text Preprocessing & Keyword Enhancement

Activity 1.2: BiLSTM Classifier (5-Class Softmax)

Activity 1.3: BERT Classifier with Class Weighting & Keyword Adjustments

Activity 1.4: Mixed Emotion Detection ($\geq 15\%$ Secondary Scores)

Activity 1.5: Unified Prediction Schema

Activity 1.6: CSV Persistence & Cached Model Loading

Milestone 4. AI-Powered Guidance and Regeneration Engine

Activity 1.1: Capture Field and Problem Context for Prompt Generation

Activity 1.2: Generate Empathetic and Context-Aware AI Responses

Activity 1.3: Response Regeneration and Synchronization Mechanism

Activity 1.4: Session HiActivity Management and CSV Logging

Milestone 1.5. Streamlit UI Implementation

Activity 1.1: Responsive Layout and Session State Management

Activity 1.2: Emotion Analysis, Model Comparison, and Visualization Components

Activity 1.3: Form Controls, Validation, and Error Handling

Activity 1.4: Analytics Dashboard and Interactive Charts

Milestone 6. User Interaction

Activity 1.1: Validate UI Flow End-to-End

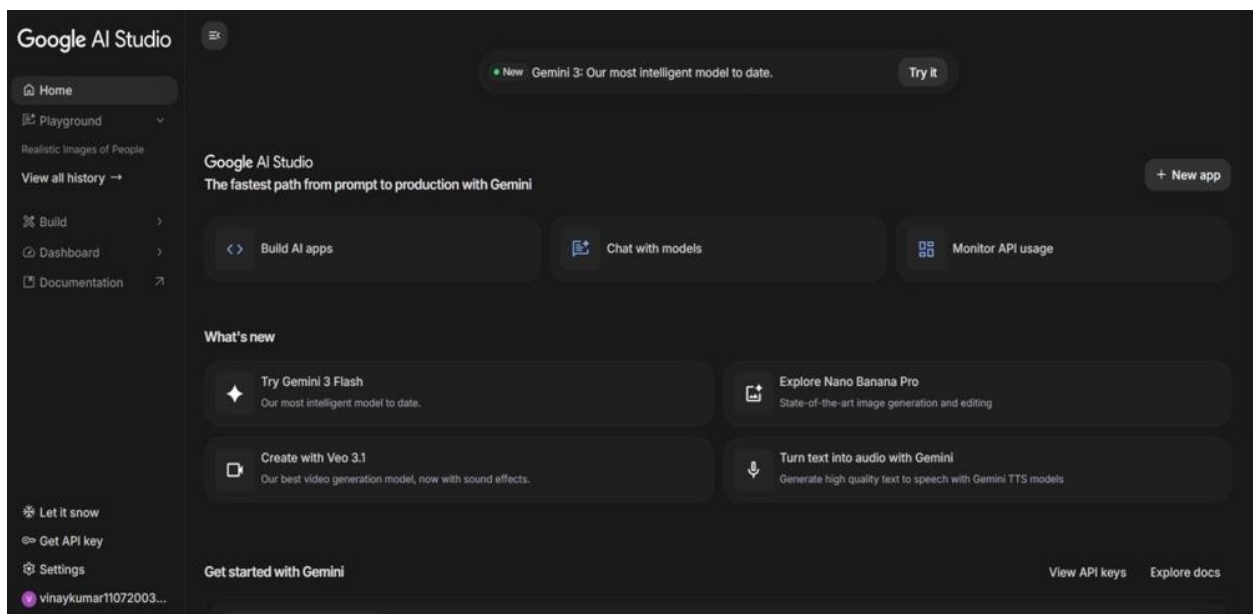
Activity 1.2: Optimization and Deployment Readiness

Milestone 1: Environment Setup and Dependency Configuration

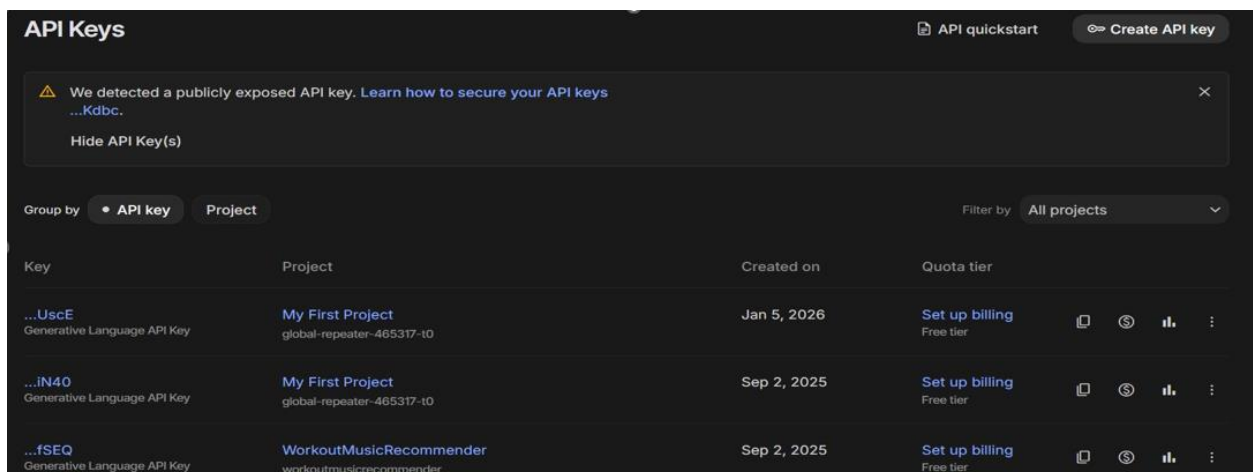
Activity 1.1: Obtain Gemini API key

Step-1: Visit the official Google AI Studio: <https://aistudio.google.com/>

1. Sign in with your Google account to access the Gemini models.

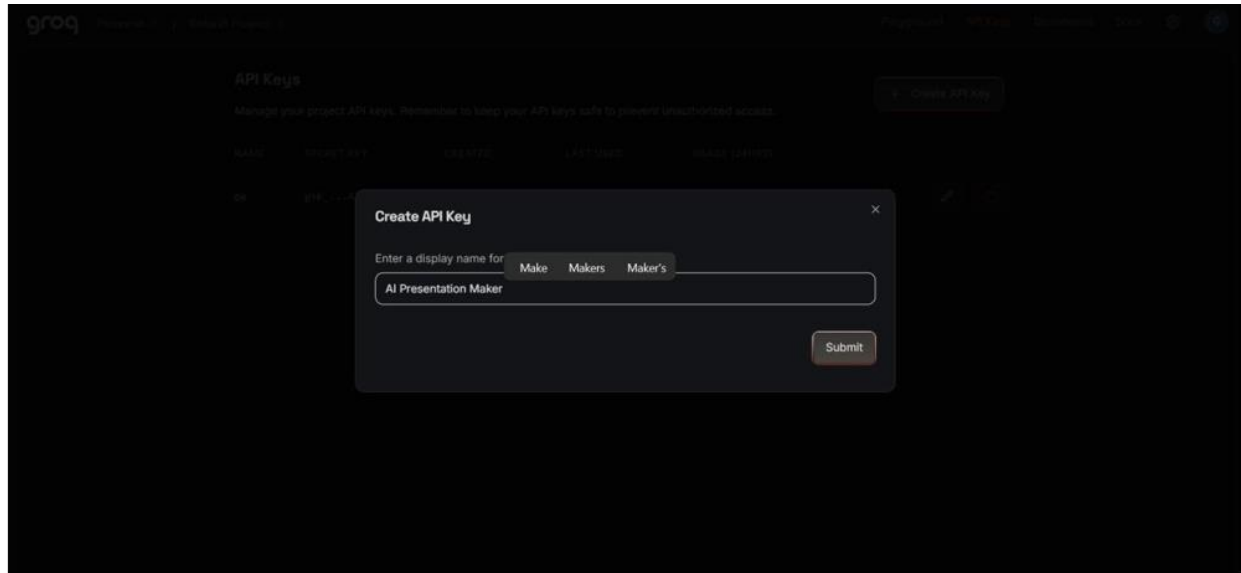


2. Click on Get API Key / Create API Key, then generate a key for your project.

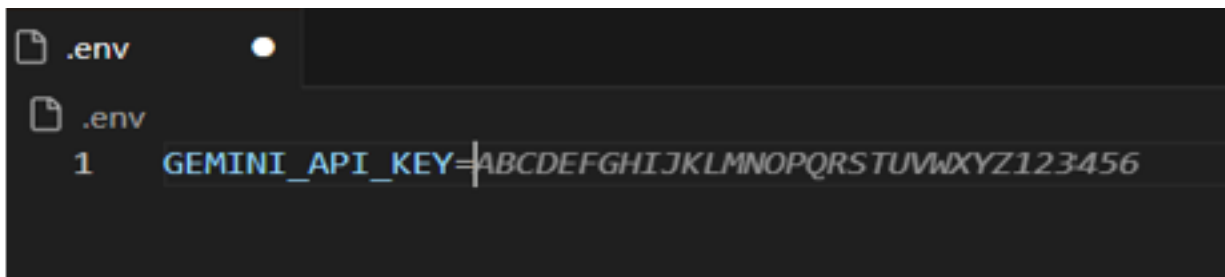


3. Navigate to the API Keys section in the dashboard.

4. Click Create API Key, then assign a recognizable name such as AI_Presentation_Maker.



5. Copy the generated key and store it securely in your project's .env file.



Activity 1.2: Install Python & Create Virtual Environment

Step 1: Open the Project Folder

1. Open Visual Studio Code.
2. Select File → Open Folder.
3. Browse to the project directory.
4. Open the project folder.

Step 2: Open the Integrated Terminal

1. Click Terminal.
2. Select New Terminal.

Alternatively, press:

Ctrl + `

The terminal opens in the project's root directory.

Step 3: Verify the Python Installation

Execute the following command:

```
python --version
```

Step 4: Create a Virtual Environment

Run the following command:

```
python -m venv venv
```

This creates a new folder named:

venv/

which contains an isolated Python environment for the project.

Step 5: Activate the Virtual Environment

Windows (Command Prompt)

```
venv\Scripts\activate
```

Windows (PowerShell)

```
.\venv\Scripts\Activate.ps1
```

macOS/Linux


```
source venv/bin/activate
```

After activation, the terminal prompt changes to:

```
(venv) C:\Emotion-Detection-Project>
```

indicating that the virtual environment is active.

Step 6: Upgrade pip

Update the package manager using:

```
python -m pip install --upgrade pip
```

Step 7: Install Project Dependencies

Install all required libraries using:

```
pip install -r requirements.txt
```

If a requirements.txt file is not yet available, install the required packages individually:

```
pip install streamlit
```

```
pip install tensorflow
```

```
pip install transformers
```

```
pip install torch
```

```
pip install scikit-learn
```

```
pip install pandas
```

```
pip install numpy
```

```
pip install nltk
```

```
pip install python-dotenv
```

```
pip install google-genai
```

```
pip install plotly
```

```
pip install openpyxl
```

Step 8: Verify Installed Packages

Run:

```
pip list
```

This displays all installed packages along with their versions.

Step 9: Select the Virtual Environment Interpreter in VS Code

1. Press:

Ctrl + Shift + P

2. Type:

Python: Select Interpreter

3. Choose the interpreter corresponding to:

venv\Scripts\python.exe

This ensures VS Code uses the project's virtual environment.

Step 10: Verify the Python Environment

Execute the following command:

```
python
```

The Python interpreter should start successfully:

```
Python 3.11.x
```

```
>>>
```

Exit the interpreter using:

```
exit()
```

Step 11: Freeze Installed Dependencies (Optional)

To save the installed package versions for reproducibility, run:

```
pip freeze > requirements.txt
```

This generates a requirements.txt file containing all project dependencies.

Expected Outcome

- A dedicated virtual environment was successfully created.
- The virtual environment was activated.
- The latest version of pip was installed.
- All required project dependencies were installed successfully.
- Visual Studio Code was configured to use the virtual environment.
- The Python environment was verified and is ready for project development.

Suggested Screenshots for the Project Report

Include screenshots of the following:

1. Project folder opened in VS Code.
2. Integrated terminal.
3. Output of `python --version`.
4. Command used to create the virtual environment.
5. Successful activation showing the (venv) prefix.
6. `pip install -r requirements.txt` (or package installation commands).
7. Output of `pip list`.
8. VS Code "Python: Select Interpreter" dialog with the virtual environment selected.
9. Python interpreter starting successfully.

These screenshots provide clear evidence that the Python development environment was set up correctly.

Activity 1.3: Install Project Dependencies

Step 1: Open the Project Folder

1. Open Visual Studio Code.
2. Open the project folder containing the source code.
3. Ensure the requirements.txt file is located in the project root directory.

Example:

Emotion-Detection-Learning-Support-Engine/

```
|
|— app.py
|— requirements.txt
|— models/
|— utils/
|— data/
|— venv/
```

Step 2: Activate the Virtual Environment

Windows (Command Prompt)

venv\Scripts\activate

Windows (PowerShell)

.\venv\Scripts\Activate.ps1

After activation, the terminal prompt should display:

(venv) C:\Emotion-Detection-Learning-Support-Engine>

Step 3: Verify the Presence of the Requirements File

Run the following command:

dir

or (Linux/macOS)

ls

Ensure that the output includes:

requirements.txt

Step 4: Install the Project Dependencies

Execute the following command in the terminal:

```
pip install -r requirements.txt
```

The installation process will automatically download and install all libraries listed in the requirements.txt file.

The dependencies include:

- TensorFlow and Keras (BiLSTM Emotion Classification)
- PyTorch and Transformers (BERT Emotion Classification)
- NLTK (Natural Language Processing)
- Pandas and NumPy (Data Processing)
- Plotly (Interactive Data Visualization)
- Streamlit (Web Application Interface)
- Google GenAI (Gemini AI Integration)
- Scikit-learn (Machine Learning Utilities)
- Python-dotenv (Environment Variable Management)
- OpenPyXL (Excel File Support)

Step 5: Wait for Installation to Complete

The installation may take several minutes depending on the internet connection and system performance.

A successful installation typically ends with a message similar to:

Successfully installed ...

Step 6: Verify the Installed Packages

Run:

```
pip list
```

This command displays all installed packages and their versions.

Verify that important libraries such as the following are present:

- tensorflow
- torch
- transformers
- streamlit
- pandas
- numpy
- nltk
- scikit-learn

- google-genai
- plotly

Step 7: Verify Package Compatibility

Check for dependency conflicts using:
`pip check`

If no issues are found, the terminal displays:
No broken requirements found.

Step 8: Verify the Installation

Run the following command to ensure the dependencies have been installed correctly:
`python app.py`

or, if using Streamlit:
`streamlit run app.py`

If the application starts without any import errors, the installation has been completed successfully.

Activity 1.4: Create a .env configuration file

Step 1: Open the Project Folder

1. Open **Visual Studio Code**.
2. Open the Emotion Detection and Learning Support Engine project.
3. Verify that the project directory is loaded successfully.

Step 2: Create a New .env File

1. In the Explorer panel, right-click on the project root folder.
2. Select **New File**.
3. Name the file:
`.env`

The .env file should be created in the root directory of the project.

Example project structure:

Emotion-Detection-Learning-Support-Engine/

```
|
|— .env
|— app.py
|— requirements.txt
|— models/
|— utils/
|— data/
```

Step 3: Add the Gemini API Key

Open the .env file and add the following line:

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

Replace YOUR_GEMINI_API_KEY with the API key generated from Google AI Studio.

Example:

```
GEMINI_API_KEY=AIzaSy*****
```

Note: Do not share or upload your actual API key to public repositories. Keep it confidential.

Step 4: Save the .env File

Save the file using:

- **File** → **Save**, or
- Press **Ctrl + S**.

Step 5: Load the Environment Variables

In the main application file (app.py), import the required modules:

```
from dotenv import load_dotenv
import os
```

Load the .env file:

```
load_dotenv()
```

Retrieve the API key:

```
api_key = os.getenv("GEMINI_API_KEY")
```

Step 6: Verify the API Key

Add the following code temporarily to verify that the API key is being loaded correctly:

```
if api_key:
    print("Gemini API Key loaded successfully.")
else:
    print("API Key not found.")
```

Run the application:

```
python app.py
```

If the .env file is configured correctly, the terminal displays:

Gemini API Key loaded successfully.

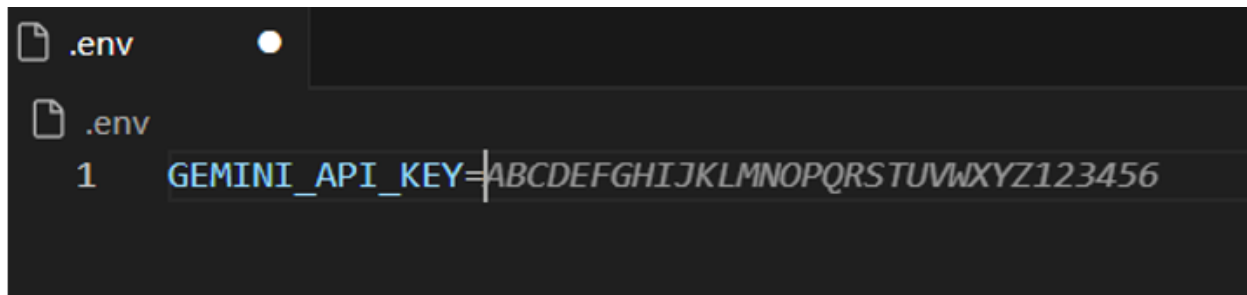
After verification, remove or comment out the print statement to avoid exposing configuration details during normal execution.

Step 7: Protect the .env File

To prevent the .env file from being uploaded to GitHub, add the following entry to the .gitignore file:

```
.env
```


This ensures that the API key remains private and is not included in version control.



```
.env
1 GEMINI_API_KEY=ABCDEFGHIJKLMNOPQRSTUVWXYZ123456
```

Activity 1.5: Verify Model and Data Directories

Step 1: Open the Project Folder

1. Open Visual Studio Code.
2. Open the Emotion Detection and Learning Support Engine project.
3. Expand the project directory in the Explorer panel.

Step 2: Verify the Model Directory

Navigate to the following folder:

Emotion-Detection-Learning-Support-Engine/

├── models/

Ensure that the required model folders are present.

Example:

```
models/
├── BiLSTM/
│   ├── emotion_model.keras
│   ├── tokenizer.pkl
│   └── label_encoder.pkl
└── BERT/
    ├── config.json
    └── model.safetensors
```

```
|— tokenizer.json
|— tokenizer_config.json
|— special_tokens_map.json
|— label_encoder.pkl
```

These folders contain the trained BiLSTM and BERT emotion classification models along with their supporting files.

Step 3: Verify the Dataset Directory

Navigate to:

```
Emotion-Detection-Learning-Support-Engine/
|
|— data/
```

Confirm that the required datasets are available.

Example:

```
data/
|
|— emotion_text_dataset.csv
|— GoEmotions/
|— EmpatheticDialogues/
|— Cleaned_Emotion_Text/
|— ISEAR/
```

These datasets are used during model training, evaluation, and testing.

Step 4: Verify Supporting Files

Ensure that the project also contains the following essential files:

```
Emotion-Detection-Learning-Support-Engine/
|
|— app.py
|— requirements.txt
|— .env
|— README.md
|— .gitignore
```

These files are required for application execution, dependency management, environment configuration, documentation, and version control.

Step 5: Verify Model Loading Paths

Open the project source code and confirm that the model paths match the directory structure.

Example:

```
emotion_predictor = EmotionPredictor(  
    model_path="models/BiLSTM/emotion_model.keras",  
    tokenizer_path="models/BiLSTM/tokenizer.pkl",  
    label_encoder_path="models/BiLSTM/label_encoder.pkl"  
)  
  
bert_predictor = BERTEmotionClassifier(  
    model_dir="models/BERT"  
)
```

Ensure that these paths correctly point to the corresponding model files.

Step 6: Verify Dataset Paths

Check that dataset paths referenced in the application are correct.

Example:

```
dataset_path = "data/emotion_text_dataset.csv"
```

If the dataset is stored in a different location, update the path accordingly.

Step 7: Run the Application

Execute the application to verify that all directories are configured correctly.

For a standard Python application:

```
python app.py
```

For a Streamlit application:

```
streamlit run app.py
```

If the application starts successfully without any file or directory errors, the directory structure has been configured correctly.

Step 8: Resolve Missing Resources (If Required)

If any model or dataset is missing:

- Copy the exported BiLSTM model into the models/BiLSTM/ folder.
- Copy the exported BERT model into the models/BERT/ folder.
- Place all required datasets inside the data/ folder.
- Verify that the filenames exactly match those referenced in the application code.

Activity 1.6: Prepare Project Folder Structure

Step 1: Create the Project Root Folder

1. Open **Visual Studio Code**.
2. Create or open the project directory.

Example:

Emotion-Detection-Learning-Support-Engine/

This folder will serve as the root directory for all project files and subdirectories.

Step 2: Create the Main Project Directories

Inside the project root folder, create the following directories:

Emotion-Detection-Learning-Support-Engine/

```
|  
|— data/  
|— models/  
|— notebooks/  
|— src/
```

Each directory is responsible for storing a specific category of project resources.

Step 3: Organize the Model Files

Inside the **models** directory, create separate folders for each trained model.

Example:

```
models/  
├──  
├── BiLSTM/  
└── BERT/
```

Store the corresponding trained model files inside their respective folders.

Step 4: Organize the Dataset Files

Place all datasets inside the **data** directory.

Example:

```
data/  
├──  
├── emotion_text_dataset.csv  
├── GoEmotions/  
├── EmpatheticDialogues/  
├── ISEAR/  
└── Cleaned_Emotion_Text/
```

This centralizes all datasets used for training, validation, and testing.

Step 5: Organize the Source Code

Create the **src** directory to store all Python modules related to the project.

Example:

```
src/  
├──  
├── preprocessing.py  
├── model.py  
├── bert_model.py  
├── predict.py  
└── train.py
```

These modules handle data preprocessing, model training, prediction, and other core functionalities.

Step 6: Add Configuration and Application Files

Store the main configuration and execution files in the project root directory.

Example:

Emotion-Detection-Learning-Support-Engine/

```
|
|— .env
|— app.py
|— requirements.txt
|— README.md
|— .gitignore
```

These files manage application execution, environment variables, project dependencies, documentation, and version control.

Step 7: Add Project Documentation and Supporting Files

Include documentation and generated output files in the project root.

Example:

Emotion-Detection-Learning-Support-Engine/

```
|
|— PROJECT_ANALYSIS_REPORT.md
|— emotion_response_examples.csv
|— emotion_response_mapping.csv
```

These files store project documentation and response data generated by the application.

Step 8: Verify the Final Folder Structure

Ensure that the completed project structure resembles the following:

Emotion-Detection-Learning-Support-Engine/

```
|
|— data/
|   |— emotion_text_dataset.csv
|   |— GoEmotions/
|   |— EmpatheticDialogues/
```

```
|
|  └── ISEAR/
|  └── Cleaned_Emotion_Text/
|
|  └── models/
|      ├── BiLSTM/
|      └── BERT/
|
|  └── notebooks/
|
|  └── src/
|      ├── preprocessing.py
|      ├── model.py
|      ├── bert_model.py
|      ├── predict.py
|      └── train.py
|
|  └── .env
|  └── .gitignore
|  └── app.py
|  └── requirements.txt
|  └── README.md
|  └── PROJECT_ANALYSIS_REPORT.md
|  └── emotion_response_examples.csv
|  └── emotion_response_mapping.csv
```

Verify that all files and folders are correctly placed before proceeding with project development.

Step 9: Open the Project in Visual Studio Code

1. Open **Visual Studio Code**.
2. Select **File** → **Open Folder**.
3. Choose the project root directory.
4. Verify that the folder hierarchy is displayed correctly in the Explorer panel.

✓ **NEW FOLDER**

>  __pycache__

>  data

✓  models

>  bert_emotion_model_final

>  bltstm

>  notebooks

✓  src

>  __pycache__

 bert_model.py

 model.py

 predict.py

 preprocessing.py

 train.py

 .env

 app.py

 emotion_response_examples.csv

 emotion_response_mapping.csv

 PROJECT_ANALYSIS_REPORT.md

 README.md

 requirements.txt

Milestone 2. Kaggle Model Training and Integration

Activity 1.1: Kaggle Setup (GPU, Dependencies, and Data Loading)

Step 1: Create a New Kaggle Notebook

1. Log in to your Kaggle account.
2. Navigate to the **Code** section.
3. Click **New Notebook** to create a new notebook.
4. Wait for the notebook environment to initialize.

Step 2: Enable GPU Acceleration

1. Open the notebook **Settings** panel.
2. Under **Accelerator**, select **GPU**.
3. Save the notebook settings.
4. Wait for the GPU runtime to initialize.

Enabling GPU acceleration significantly reduces the training time required for deep learning models such as BiLSTM and BERT.

Step 3: Install the Required Dependencies

Install all required libraries by executing the installation cell in the Kaggle notebook.

The installed packages include:

- TensorFlow 2.17.0
- NumPy 1.26.4
- Pandas
- Scikit-learn
- SciPy
- Matplotlib
- Seaborn

- NLTK
- Transformers
- PyTorch
- Plotly
- Protobuf

These libraries provide support for:

- Deep learning model development
- Natural language processing
- Data preprocessing
- Model evaluation
- Visualization
- Transformer-based architectures

Wait until all packages are installed successfully before proceeding to the next step.

Step 4: Verify the Installed Libraries

After the installation is complete, import the required libraries to verify that they have been installed correctly.

Verify the following libraries:

- TensorFlow
- NumPy
- Scikit-learn
- Transformers

The notebook should display the installed version number for each library.

Example verification includes:

- TensorFlow Version
- NumPy Version

- Scikit-learn Version
- Transformers Version

Successful version outputs confirm that the environment has been configured correctly.

Step 5: Verify GPU Availability

Execute the GPU verification cell.

The notebook checks whether TensorFlow detects an available GPU device.

If GPU acceleration is enabled correctly, the notebook displays:

- Number of GPUs detected
- GPU device information

This confirms that the deep learning models will utilize GPU resources during training.

Step 6: Verify Environment Stability

After dependency installation and GPU detection, confirm that:

- All required libraries were imported successfully.
- No dependency conflicts occurred.
- GPU detection completed successfully.
- TensorFlow recognizes the GPU device.

The notebook should indicate that the Kaggle environment is stable and ready for model training.

Step 7: Load the Emotion Datasets

Load all datasets required for model training into the notebook.

The datasets include:

- GoEmotions
- EmpatheticDialogues
- ISEAR

- Emotion Text Dataset
- Cleaned Emotion Context Dataset

Verify that all datasets are accessible and located in the correct directory.

Step 8: Verify Dataset Loading

Inspect each dataset after loading to ensure that:

- The dataset loads without errors.
- All required columns are present.
- The records are displayed correctly.
- Missing or corrupted files are not detected.

This verification confirms that the datasets are ready for preprocessing and model training.

Activity 1.2: Data Preprocessing & Tokenization

Step 1: Load the Emotion Datasets

Load the emotion datasets into the Kaggle notebook using the Pandas library.

Verify that each dataset is loaded successfully and contains the expected number of records.

The datasets include:

- GoEmotions
- EmpatheticDialogues
- ISEAR
- Emotion Text Dataset
- Cleaned Emotion Context Dataset

Step 2: Merge the Datasets

Combine all emotion datasets into a single unified dataset.

During this process:

- Standardize the column names.
- Ensure consistent text formatting.
- Merge all records into one dataset.
- Remove duplicate entries if present.

The unified dataset serves as the primary dataset for model training.

Step 3: Handle Missing Values

Inspect the dataset for missing or null values.

Perform the following operations:

- Remove empty text entries.
- Remove records with missing emotion labels.
- Replace invalid values where necessary.
- Verify that the cleaned dataset contains only valid records.

This step improves the quality and reliability of the training data.

Step 4: Clean the Text Data

Clean each text sample to remove unnecessary characters while preserving emotional meaning.

The preprocessing operations include:

- Convert all text to lowercase.
- Remove URLs.
- Remove HTML tags.
- Remove punctuation marks.
- Remove special characters.
- Remove extra whitespace.
- Remove unwanted symbols.
- Preserve meaningful emotional words.

This ensures that the models receive consistent and noise-free input.

Step 5: Normalize the Text

Normalize the cleaned text by applying standard formatting techniques.

Typical normalization steps include:

- Expand contractions where applicable.
- Normalize repeated spaces.
- Standardize text formatting.
- Preserve contextual information relevant to emotion detection.

Normalization helps maintain consistency across all text samples.

Step 6: Perform Tokenization

Convert each cleaned sentence into individual tokens.

For the BiLSTM model:

- Create a tokenizer.
- Build the vocabulary from the training data.
- Convert each sentence into a sequence of integer tokens.

For the BERT model:

- Apply the pre-trained BERT tokenizer.
- Generate input IDs.
- Generate attention masks.
- Preserve the contextual representation of each sentence.

Tokenization transforms textual data into a machine-readable format.

Step 7: Apply Padding and Sequence Generation

Since sentences have varying lengths, convert them into fixed-length sequences.

Perform the following operations:

- Pad shorter sequences with zeros.
- Truncate sequences that exceed the maximum length.
- Maintain a consistent sequence length for all samples.

This ensures that all inputs have a uniform shape during model training.

Step 8: Encode Emotion Labels

Convert the categorical emotion labels into numerical values.

The encoding process assigns a unique integer to each emotion class.

For example:

- Anxious
- Bored
- Confident
- Confused
- Curious
- Frustrated

The encoded labels are then used as target outputs during model training.

Step 9: Split the Dataset

Divide the processed dataset into training and testing sets.

A typical split includes:

- Training dataset
- Validation dataset
- Testing dataset

This enables the models to learn from one portion of the data while evaluating performance on unseen samples.

Step 10: Verify the Preprocessed Dataset

Review the final dataset to ensure that:

- All text has been cleaned successfully.
- Tokenization completed without errors.
- Padding has been applied correctly.
- Labels have been encoded successfully.
- No missing values remain.
- The dataset is ready for deep learning model training.

Activity 1.3: BiLSTM Model Training

Step 1: Import the Required Libraries

Import all libraries required for model development and training.

The required libraries include:

- TensorFlow
- Keras
- NumPy
- Scikit-learn
- Matplotlib

These libraries provide support for building, training, and evaluating the BiLSTM model.

Step 2: Load the Preprocessed Dataset

Load the preprocessed training, validation, and testing datasets.

Verify that:

- Input sequences are padded to a fixed length.
- Emotion labels are correctly encoded.
- Dataset dimensions match the expected input shape.

Step 3: Define the BiLSTM Model Architecture

Construct the BiLSTM model using TensorFlow/Keras.

The architecture typically consists of:

- Input Layer
- Embedding Layer
- Bidirectional LSTM Layer
- Dropout Layer
- Dense Layer
- Softmax Output Layer

The embedding layer converts tokenized words into dense vector representations, while the Bidirectional LSTM processes the text in both forward and backward directions to capture contextual information.

Step 4: Configure the Model Parameters

Configure the hyperparameters required for training.

Typical parameters include:

- Vocabulary size
- Maximum sequence length
- Embedding dimension
- Number of LSTM units
- Batch size
- Number of epochs
- Learning rate
- Dropout rate

These parameters control the learning process and influence the model's performance.

Step 5: Compile the Model

Compile the BiLSTM model by specifying:

- Loss function
- Optimizer
- Evaluation metrics

For multiclass emotion classification:

- Loss Function: Categorical Cross-Entropy (or Sparse Categorical Cross-Entropy)
- Optimizer: Adam
- Metric: Accuracy

Compiling prepares the model for the training process.

Step 6: Train the BiLSTM Model

Train the model using the prepared training dataset.

During training:

- The model learns from the training data.
- Validation accuracy and validation loss are monitored after each epoch.
- Model weights are updated to improve prediction accuracy.

GPU acceleration significantly reduces the training time.

Step 7: Monitor Training Performance

Observe the training progress after every epoch.

The following metrics should be monitored:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss

These metrics help determine whether the model is learning effectively and whether overfitting or underfitting is occurring.

Step 8: Evaluate the Trained Model

After training is complete, evaluate the model using the testing dataset.

Calculate evaluation metrics such as:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

These metrics provide a comprehensive assessment of the model's classification performance.

Step 9: Save the Trained Model

Save the trained BiLSTM model and its supporting files for future inference.

Store the following files:

- Trained BiLSTM model
- Tokenizer
- Label Encoder

Save them inside the project directory:

```
models/  
└── BiLSTM/  
    ├── emotion_model.keras  
    ├── tokenizer.pkl  
    └── label_encoder.pkl
```

Saving these files allows the trained model to be reused without retraining.

Step 10: Verify Model Saving

Confirm that:

- The model file has been saved successfully.
- The tokenizer file is available.
- The label encoder has been stored correctly.
- The files can be loaded **without errors**.

This verification ensures that the model is ready for integration into the application.

Activity 1.4: Domain-Adaptive Fine-Tuning

Step 1: Load the Pre-trained BiLSTM Model

Load the previously trained BiLSTM model into the Kaggle notebook.

Ensure that the following files are available:

- Trained BiLSTM model
- Tokenizer
- Label encoder

Verify that the model loads successfully without any errors.

Step 2: Load the Domain-Specific Dataset

Load the student-specific emotion dataset prepared for domain adaptation.

Verify that:

- The dataset is successfully loaded.
- Emotion labels are correctly encoded.
- Text samples have already been cleaned and tokenized.
- Input sequences match the format used during the initial training process.

Step 3: Prepare the Fine-Tuning Dataset

Prepare the dataset for fine-tuning by performing the following tasks:

- Shuffle the dataset to improve training randomness.
- Split the dataset into training and validation sets.
- Verify that the input sequences and emotion labels are correctly aligned.

This ensures that the model learns effectively from the domain-specific data.

Step 4: Configure Fine-Tuning Parameters

Configure the training parameters for domain adaptation.

Typical parameters include:

- Learning rate
- Batch size
- Number of fine-tuning epochs
- Validation split
- Optimizer
- Loss function

A lower learning rate is generally used during fine-tuning to preserve the knowledge learned during the initial training while adapting the model to the new domain.

Step 5: Initialize Fine-Tuning

Initialize the training process using the pre-trained BiLSTM model and the domain-specific dataset.

During fine-tuning:

- Existing model weights are retained.
- Model parameters are gradually updated using the new training data.
- The model adapts to educational and student-related emotional expressions.

Step 6: Monitor the Fine-Tuning Process

Observe the training progress throughout each epoch.

Monitor the following performance metrics:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss

These metrics help evaluate whether the model is successfully adapting to the new dataset without overfitting.

Step 7: Evaluate the Fine-Tuned Model

After fine-tuning is complete, evaluate the updated model using the testing dataset.

Calculate the following evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Compare these results with those obtained from the original BiLSTM model to assess the improvements achieved through domain adaptation.

Step 8: Save the Fine-Tuned Model

Save the updated BiLSTM model after successful fine-tuning.

Store the following files:

- Fine-tuned BiLSTM model
- Updated tokenizer (if modified)
- Updated label encoder (if modified)

Example directory structure:

```
models/  
└── BiLSTM/  
    ├── bilstm_student_adaptive.keras  
    ├── tokenizer.pkl  
    └── label_encoder.pkl
```

Saving these files enables the fine-tuned model to be reused during application deployment.

Step 9: Verify Model Saving

Confirm that:

- The fine-tuned model has been saved successfully.
- The saved model can be reloaded without errors.
- The tokenizer and label encoder are available.
- The model is ready for integration into the application.

Step 10: Compare Model Performance

Compare the performance of the original BiLSTM model with the fine-tuned model.

Evaluate improvements based on:

- Overall accuracy
- Precision
- Recall
- F1-score
- Emotion-wise classification accuracy

Document any observed improvements in recognizing student-specific emotions after domain adaptation.

Activity 1.5: BERT Model Fine-Tuning

Step 1: Load the Pre-trained BERT Model

Load the pre-trained bert-base-uncased model along with its tokenizer using the Hugging Face Transformers library. Verify that both the model and tokenizer are loaded successfully.

Step 2: Prepare the Training Dataset

Convert the cleaned text into BERT input features by generating:

- Input IDs
- Attention Masks
- Encoded Emotion Labels

Create training, validation, and testing datasets suitable for BERT training.

Step 3: Configure Training Parameters

Configure the training settings for the model, including:

- Learning Rate
- Batch Size
- Number of Epochs
- Optimizer (AdamW)
- Weight Decay
- Evaluation Strategy

These parameters are selected to achieve stable convergence while minimizing overfitting.

Step 4: Fine-Tune the BERT Model

Train the BERT model using the prepared training dataset.

During training:

- Forward propagation computes the predictions.
- Backpropagation updates the model weights.
- Validation is performed after each epoch to monitor learning progress.

GPU acceleration significantly reduces the training time.

Step 5: Monitor Model Performance

Monitor the following metrics throughout the training process:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

These metrics help determine whether the model is learning effectively and generalizing well to unseen data.

Step 6: Evaluate the Fine-Tuned Model

After training, evaluate the model using the testing dataset.

Calculate the following evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Analyze the results to measure the effectiveness of the fine-tuned BERT model.

Step 7: Save the Fine-Tuned Model

Save the trained BERT model and all associated files for deployment.

Store the following files:

- Model Weights
- Configuration File

- Tokenizer Files
- Label Encoder

Example directory:

```
models/  
└── BERT/  
    ├── config.json  
    ├── model.safetensors  
    ├── tokenizer.json  
    ├── tokenizer_config.json  
    ├── special_tokens_map.json  
    └── label_encoder.pkl
```

Step 8: Verify the Saved Model

Reload the saved BERT model and tokenizer to verify that:

- The model loads successfully.
- The tokenizer functions correctly.
- Sample predictions can be generated without errors.

Successful verification confirms that the model is ready for integration into the Emotion Detection and Learning Support Engine.

Activity 1.6: Model Exportation & Local Integration

Step 1: Export the Trained Models

After training, export the following model files from the Kaggle notebook:

- BiLSTM model (emotion_model.keras)
- BERT model files (config.json, model.safetensors, tokenizer files)
- Tokenizer
- Label Encoder

Download the exported files to the local system.

Step 2: Organize the Model Files

Create separate directories for each model inside the local project and copy the exported files into their respective folders.

Example directory structure:

```
models/
├── BiLSTM/
│   ├── emotion_model.keras
│   ├── tokenizer.pkl
│   └── label_encoder.pkl
└── BERT/
    ├── config.json
    ├── model.safetensors
    ├── tokenizer.json
    ├── tokenizer_config.json
    ├── special_tokens_map.json
    └── label_encoder.pkl
```

Step 3: Configure Model Loading

Update the application to load the exported models using the correct file paths. Ensure that the BiLSTM model, BERT model, tokenizer, and label encoder are initialized successfully when the application starts.

Step 4: Verify Local Integration

Run the application and load both models. Perform sample emotion predictions using test inputs to confirm that the models produce valid outputs without any file path or loading errors.

Step 5: Validate the Integration

Verify that:

- Both models load successfully.
- Tokenizers and label encoders function correctly.
- Predictions are generated without errors.
- The application is ready for the next development phase.

Milestone 3: Core Emotion Detection Pipeline Development

Activity 1.1: Text Preprocessing and Keyword Enhancement

Step 1: Receive User Input

Accept the student's text input through the application's user interface.

Validate that:

- The input field is not empty.
- The entered text is in a valid format.
- Invalid or blank inputs are handled appropriately.

Step 2: Clean the Input Text

Perform text preprocessing to remove unnecessary information from the user's input.

The preprocessing operations include:

- Convert text to lowercase.
- Remove URLs.
- Remove HTML tags.
- Remove punctuation.
- Remove special characters.
- Remove extra whitespace.
- Remove unwanted symbols.

This produces a clean and standardized text sample for further analysis.

Step 3: Normalize the Text

Normalize the cleaned text to improve consistency.

Typical normalization includes:

- Expanding contractions where applicable.
- Standardizing word formatting.
- Removing repeated spaces.
- Preserving meaningful emotional expressions.

This step ensures uniform input across all predictions.

Step 4: Remove Stop Words

Filter out common stop words that contribute little to emotion classification.

Examples include:

- is
- the
- a
- an
- and
- of
- in

Removing stop words allows the models to focus on meaningful emotional content.

Step 5: Perform Tokenization

Convert the processed text into numerical tokens.

For the BiLSTM model:

- Convert words into integer sequences.
- Apply sequence padding to match the required input length.

For the BERT model:

- Generate Input IDs.
- Generate Attention Masks.

- Prepare the input in the format required by the transformer model.

Step 6: Apply Keyword Enhancement

Analyze the processed text to identify keywords that indicate specific student emotions.

Examples include keywords related to:

- Confusion
- Curiosity
- Confidence
- Frustration
- Boredom
- Anxiety

The identified keywords are used to strengthen the contextual understanding of the input before prediction.

Step 7: Adjust Emotion Weights

Based on the detected keywords, apply predefined weight adjustments to the predicted emotion probabilities.

This process helps the system better recognize educational expressions and improves the prediction of emotions commonly observed in learning environments.

Step 8: Generate the Enhanced Input

Combine the cleaned text, tokenized representation, and keyword-enhanced information into the final input format required by the prediction models.

Verify that:

- Input sequences are valid.
- Required tensors are generated correctly.
- No preprocessing errors occur.

Step 9: Validate the Preprocessing Pipeline

Test the preprocessing pipeline using multiple student text samples.

Confirm that:

- Text cleaning is performed correctly.
- Keywords are detected accurately.
- Tokenization is successful.
- The processed input is accepted by both the BiLSTM and BERT models.

Activity 1.2: BiLSTM Classifier (5-Class Softmax)

Step 1: Load the Trained BiLSTM Model

Load the trained BiLSTM model during application startup.

Verify that:

- The model file loads successfully.
- The tokenizer is initialized correctly.
- The label encoder is loaded without errors.

This ensures that the classifier is ready to process incoming text.

Step 2: Receive the Preprocessed Text

Accept the cleaned and tokenized text generated by the preprocessing pipeline.

Ensure that:

- The input text is valid.
- Empty inputs are rejected.
- The text format matches the model's expected input.

Step 3: Convert Text into Numerical Sequences

Transform the processed text into numerical sequences using the trained tokenizer.

Apply sequence padding to ensure that all inputs have a fixed length before being passed to the model.

Step 4: Perform Emotion Prediction

Provide the padded input sequence to the BiLSTM model.

The model analyzes the contextual relationships within the sentence and predicts the probability of each emotion class.

Step 5: Calculate Prediction Confidence

Obtain the probability scores produced by the classifier.

Identify:

- The predicted emotion.
- The confidence score associated with the prediction.
- The probability distribution across all supported emotion classes.

Step 6: Apply Post-processing

Convert the numerical prediction into its corresponding emotion label.

If keyword enhancement has been applied, incorporate the adjusted probabilities before selecting the final predicted emotion.

This step improves the interpretability of the classifier's output.

Step 7: Handle Low-Confidence Predictions

Evaluate the confidence score produced by the model.

If the confidence value falls below the predefined threshold:

- Flag the prediction as uncertain.
- Allow further analysis by the BERT classifier.
- Continue the prediction pipeline without interrupting the application.

This approach improves the robustness of the overall system.

Step 8: Return the Prediction Results

Generate the final prediction output containing:

- Predicted emotion.
- Confidence score.
- Emotion probability scores.
- Processed input text.

These results are forwarded to the subsequent components of the Emotion Detection and Learning Support Engine.

Step 9: Verify Classifier Performance

Test the classifier using multiple student text samples.

Verify that:

- Predictions are generated successfully.
- Confidence scores are displayed correctly.
- Emotion labels match the expected outputs.
- No runtime errors occur during inference.

Activity 1.3: BERT Classifier with Class Weighting & Keyword Adjustments

Step 1: Load the Fine-Tuned BERT Model

Load the fine-tuned BERT model during application startup.

Verify that:

- The model loads successfully.
- The tokenizer is initialized correctly.
- The label encoder is available.
- All required model files are accessible.

This prepares the classifier for emotion prediction.

Step 2: Receive the Processed Input

Accept the cleaned and preprocessed text generated by the preprocessing module.

Ensure that the input:

- Contains valid text.
- Has been tokenized correctly.
- Matches the format required by the BERT model.

Step 3: Generate BERT Input Features

Convert the processed text into BERT-compatible input features.

Generate:

- Input IDs
- Attention Masks
- Token Type IDs (if required)

These features allow the BERT model to process contextual relationships within the text.

Step 4: Perform Emotion Prediction

Pass the generated input features to the BERT classifier.

The model predicts the probability of each supported emotion class by analyzing the contextual meaning of the student's text.

Step 5: Apply Class Weighting

Adjust the predicted probabilities using predefined class weights.

Class weighting reduces prediction bias and improves the recognition of emotion classes that occur less frequently within the training dataset.

This helps produce more balanced and reliable predictions across all supported emotions.

Step 6: Apply Keyword-Based Adjustments

Analyze the processed text for predefined emotion-related keywords.

If significant keywords are detected, adjust the corresponding emotion probabilities according to the predefined keyword enhancement rules.

This improves recognition of educational expressions such as confusion, curiosity, confidence, frustration, boredom, and anxiety.

Step 7: Generate the Final Emotion Prediction

Combine the contextual predictions produced by BERT with the adjusted probability scores.

Determine:

- Final predicted emotion.
- Confidence score.
- Probability distribution for all emotion classes.

The emotion with the highest adjusted probability is selected as the final prediction.

Step 8: Return the Classification Results

Return the prediction results to the application.

The output should include:

- Predicted emotion
- Confidence score
- Emotion probability scores
- Processed input text

These results are forwarded to the subsequent stages of the Emotion Detection and Learning Support Engine.

Step 9: Validate the Classifier

Test the classifier using multiple student text samples.

Verify that:

- The model loads successfully.
- Predictions are generated without errors.
- Class weighting is applied correctly.
- Keyword adjustments improve prediction quality.
- Confidence scores are displayed correctly.

Successful validation confirms that the classifier is functioning as expected.

Activity 1.4: Mixed-Emotion Detection ($\geq 15\%$ Secondary Scores)

Step 1: Receive Emotion Probability Scores

Obtain the probability scores generated by the emotion classification models.

Ensure that a probability value is available for each supported emotion class before performing mixed emotion analysis.

Step 2: Identify the Primary Emotion

Determine the emotion with the highest probability score.

This emotion is considered the primary emotion and serves as the baseline for further mixed emotion analysis.

Step 3: Define the Confidence Threshold

Set a predefined confidence threshold for identifying additional emotions.

Only emotions with probability scores greater than or equal to the selected threshold are considered as part of the final mixed emotion result.

This prevents low-confidence predictions from affecting the final outcome.

Step 4: Detect Secondary Emotions

Compare the probability scores of all remaining emotion classes against the predefined threshold.

If one or more additional emotions satisfy the threshold, include them as secondary emotions.

This enables the system to recognize situations where multiple emotions coexist within the same student response.

Step 5: Rank the Detected Emotions

Arrange the detected emotions in descending order according to their probability scores.

The ranking indicates the relative strength of each detected emotion and provides a clear interpretation of the student's emotional state.

Step 6: Generate the Mixed Emotion Output

Create the final prediction by combining the primary emotion with all detected secondary emotions.

The generated output should include:

- Primary emotion
- Secondary emotion(s)
- Confidence score for each detected emotion
- Overall emotion probability distribution

This output provides a comprehensive representation of the student's emotions.

Step 7: Handle Single Emotion Cases

If no additional emotions exceed the predefined threshold, classify the input as containing a single dominant emotion.

This ensures that the system can accurately distinguish between single-emotion and mixed-emotion scenarios.

Step 8: Display the Prediction Results

Present the detected emotions within the application interface.

Display:

- Primary emotion
- Secondary emotion(s), if any
- Confidence score for each detected emotion
- Emotion probability scores

The results should be displayed in a clear and user-friendly format.

Step 9: Validate the Mixed Emotion Detection Module

Test the module using multiple student text samples representing both single-emotion and mixed-emotion situations.

Verify that:

- Multiple emotions are detected correctly when applicable.
- Confidence thresholds are applied accurately.
- Emotion rankings are generated correctly.
- The final output reflects the emotional content of the input text.

Successful validation confirms the reliability of the mixed emotion detection process.

Activity 1.5: Unified Prediction Schema

Step 1: Collect Prediction Results

Receive the prediction outputs generated by the BiLSTM classifier, BERT classifier, and mixed emotion detection module.

Verify that all required prediction values are available before proceeding.

Step 2: Standardize the Output Format

Convert the outputs from all prediction modules into a common format.

Include the following information:

- Predicted emotion
- Mixed emotions (if detected)
- Confidence score
- Emotion probability scores
- Model information

This ensures consistency across all prediction results.

Step 3: Merge Prediction Data

Combine the standardized outputs into a single prediction object.

Ensure that duplicate or conflicting information is resolved and that the final prediction accurately represents the analysis performed by the system.

Step 4: Validate the Unified Prediction

Verify that the combined prediction contains complete and accurate information.

Check that:

- Emotion labels are valid.
- Confidence scores are correctly calculated.
- Mixed emotion results are included when applicable.
- No prediction fields are missing.

Step 5: Return the Unified Prediction

Provide the unified prediction to the next module of the application for response generation, visualization, logging, and report creation.

The prediction should be returned in a structured format that can be easily processed throughout the system.

Activity 1.6: CSV Persistence & Cached Model Loading

Step 1: Create CSV Storage Files

Create CSV files to store prediction results and interaction history.

Typical information stored includes:

- Timestamp
- User input
- Predicted emotion
- Confidence score
- AI-generated response
- Subject or field of study

These files maintain a record of user interactions for future reference.

Step 2: Implement Data Persistence

Configure the application to automatically append each prediction to the corresponding CSV file after successful emotion analysis.

Verify that new records are saved without overwriting existing data.

Step 3: Enable Cached Model Loading

Configure cached loading for the BiLSTM and BERT models during application startup.

The models are loaded once and reused for subsequent predictions, reducing repeated initialization and improving execution speed.

Step 4: Validate Stored Data

Open the generated CSV files and verify that:

- Each prediction is recorded correctly.
- Data fields are complete.
- Timestamps are generated accurately.

- No duplicate or corrupted records are present.

Step 5: Verify Application Performance

Run the application multiple times and confirm that:

- Models are loaded from cache successfully.
- Prediction time is reduced after the initial load.
- User interactions continue to be saved correctly in the CSV files.
- The application operates without storage or loading errors.

Milestone 4: AI-Powered Guidance and Regeneration Engine

Activity 1.1: Capture Field + Problem, Build Gemini Prompt with Emotion/Confidence

Step 1: Capture User Inputs

Collect the following information from the application interface:

- Selected academic field
- Student's problem description

Validate that both fields contain valid input before proceeding.

Step 2: Retrieve Emotion Prediction

Receive the prediction results generated by the emotion detection module.

Extract the following information:

- Predicted emotion
- Confidence score
- Mixed emotions (if detected)

Ensure that the prediction data is available for prompt generation.

Step 3: Combine Input Information

Merge the user's academic field, problem description, detected emotion, and confidence score into a structured data object.

This creates a complete context describing both the student's learning issue and emotional state.

Step 4: Construct the Gemini Prompt

Generate a structured prompt containing:

- Academic field
- Student's problem
- Detected emotion
- Confidence score
- Instruction requesting supportive and personalized learning guidance

The prompt should clearly instruct Gemini to provide an encouraging, subject-specific, and emotionally appropriate response.

Step 5: Validate the Prompt

Verify that the generated prompt:

- Includes all required information.
- Contains no missing or invalid fields.
- Accurately represents the student's context.
- Is properly formatted before being sent to Gemini.

Step 6: Forward the Prompt to Gemini

Send the validated prompt to the Gemini API for response generation.

The model analyzes the combined information and generates personalized learning support tailored to the student's academic field and emotional condition.

Activity 1.2: Generate Empathetic, Field-Aware Responses; Fallback to Templates

Step 1: Receive the Gemini Prompt

Obtain the structured prompt generated from:

- Academic field
- Student's problem description
- Predicted emotion
- Confidence score

Verify that the prompt is complete before submitting it to the Gemini model.

Step 2: Generate an AI Response

Send the prompt to the Gemini API.

The model analyzes the student's academic context and emotional state to generate a personalized learning response that is:

- Empathetic
- Subject-specific
- Encouraging
- Action-oriented

Step 3: Validate the Generated Response

Verify that the AI response:

- Is successfully returned.
- Is not empty or incomplete.
- Addresses the student's learning problem.
- Matches the detected emotion and academic field.

If validation succeeds, display the response to the user.

Step 4: Apply the Fallback Mechanism

If the Gemini API is unavailable or fails to generate a valid response:

- Detect the failure automatically.
- Retrieve the predefined response template corresponding to the predicted emotion.
- Generate a supportive response using the stored template.

This ensures uninterrupted assistance even when AI generation is unavailable.

Step 5: Display the Final Response

Present the final learning guidance within the application.

The displayed response should include:

- Personalized learning advice.
- Emotionally supportive language.
- Subject-specific recommendations.
- Practical suggestions for improving understanding.

Activity 1.3: Regenerate Responses When Input/AI Toggle Changes; Keep Scores in Sync

Step 1: Monitor User Interactions

Continuously monitor changes to:

- Student problem description.
- Selected academic field.
- AI response toggle.
- Analyze or prediction request.

Detect any modification that requires the prediction results to be refreshed.

Step 2: Reprocess the Updated Input

Whenever a change is detected:

- Validate the updated input.
- Execute the text preprocessing pipeline.
- Perform keyword enhancement.
- Prepare the input for the prediction models.

This ensures that the latest information is used for analysis.

Step 3: Regenerate Emotion Predictions

Run the BiLSTM classifier, BERT classifier, and mixed emotion detection module using the updated input.

Generate:

- Predicted emotion.
- Mixed emotions (if applicable).
- Confidence score.
- Emotion probability scores.

Step 4: Synchronize Prediction Results

Update all displayed prediction values simultaneously.

Ensure that:

- Emotion labels.
- Confidence scores.
- Probability distributions.

- Mixed emotion results.

are refreshed together without inconsistencies.

Step 5: Regenerate the Learning Response

If the AI response toggle is enabled, generate a new personalized response using Gemini.

If the AI response toggle is disabled or AI generation is unavailable, automatically display the predefined emotion-based response template.

Step 6: Update the User Interface

Refresh the application interface with the latest:

- Emotion prediction.
- Confidence score.
- Learning response.
- Supporting analytics.

Confirm that all displayed information corresponds to the current user input and application settings.

Activity 1.4: Maintain Session History and CSV Logs for Continuous Learning

Step 1: Initialize Session History

Create a session history module to store user interactions during the current application session.

For each interaction, maintain:

- Timestamp
- Academic field

- Student problem description
- Predicted emotion
- Confidence score
- AI-generated or template response

Step 2: Record Prediction Results

After each successful emotion prediction, automatically add the interaction to the session history. Ensure that every new prediction is stored without removing previous session records.

Step 3: Update CSV Log Files

Append each interaction to the CSV log files.

Record the following information:

- Date and time
- Academic field
- User input
- Predicted emotion
- Confidence score
- Mixed emotions (if detected)
- Generated learning response

This creates a persistent record of all prediction activities.

Step 4: Synchronize Session and CSV Data

Verify that every interaction stored in the session history is also written successfully to the CSV log files.

Check for missing or duplicate records to maintain data consistency.

Step 5: Display Session History

Present the stored session history within the application.

Allow users to review previous predictions, responses, and confidence scores generated during the current session.

Step 6: Validate Continuous Learning Records

Verify that:

- New interactions are recorded successfully.
- CSV files are updated correctly.
- Session history remains synchronized with the log files.
- Historical data is available for future analysis and system improvement.

Milestone 5: Streamlit UI Implementation

Activity 1.1: Responsive Layout with Session State for Inputs/Results/Analytics

Step 1: Design the User Interface

Create a responsive application layout that organizes the interface into dedicated sections for:

- User input
- Emotion prediction results
- Learning response
- Analytics dashboard

Ensure that the layout remains clear and accessible across different screen sizes.

Step 2: Initialize Session State

Configure session state variables to preserve application data during user interactions.

Store information such as:

- Academic field
- Student problem description
- Emotion prediction
- Confidence score
- AI-generated response
- Session history

This prevents data loss during interface updates.

Step 3: Capture and Preserve User Inputs

Store user inputs within the session state whenever a prediction is performed.

Ensure that previously entered information remains available even after the application refreshes or new predictions are generated.

Step 4: Update Prediction Results

After each prediction, update the session state with the latest:

- Predicted emotion
- Mixed emotions

- Confidence score
- Emotion probability scores
- Learning response

This keeps the displayed information synchronized throughout the application.

Step 5: Refresh the Analytics Section

Automatically update the analytics dashboard using the latest session data.

Display information such as:

- Emotion distribution
- Confidence trends
- Prediction history
- Field-wise analysis

The analytics should refresh whenever new predictions are generated.

Step 6: Verify Layout Responsiveness

Test the application to ensure that:

- User inputs are preserved correctly.
- Prediction results remain synchronized.
- Analytics update automatically.
- The interface remains responsive and easy to navigate during continuous use.

Activity 1.2: Sections for field selection, problem input, model comparison (BiLSTM vs BERT), mixed emotions, confidence bars, analytics tabs.

Step 1: Create the Input Section

Design the input section to collect the following information from the user:

- Academic field or subject
- Student problem description

Validate the inputs before initiating emotion prediction.

Step 2: Display Emotion Prediction Results

Create a results section to present the predicted emotion along with the confidence score.

Ensure that the displayed information updates automatically after each prediction.

Step 3: Implement Model Comparison

Display the prediction results generated by both the BiLSTM and BERT classifiers.

Present the following information side by side for comparison:

- Predicted emotion
- Confidence score
- Model-specific prediction results

This enables users to compare the outputs of both models.

Step 4: Display Mixed Emotion Analysis

Create a dedicated section to display the mixed emotion detection results.

Show:

- Primary emotion
- Secondary emotion(s), if detected
- Confidence score for each detected emotion

This provides a more comprehensive interpretation of the student's emotional state.

Step 5: Visualize Confidence Scores

Represent the confidence scores of all supported emotion classes using graphical confidence bars.

Display the scores in descending order to improve readability and help users understand the model's prediction confidence.

Step 6: Create Analytics Tabs

Organize the analytics dashboard into separate tabs for different types of analysis, such as:

- Emotion Distribution
- Confidence Analysis
- Field-wise Predictions
- Session History Summary

Update the analytics automatically whenever new predictions are generated.

Activity 1.3: Form controls, run/clear, spinners, error handling

Step 1: Design the Form Controls

Create an interactive form containing:

- Academic field selection
- Problem description input
- AI response toggle
- Prediction settings (if applicable)

Ensure that all form fields are clearly labeled and easy to use.

Step 2: Implement the Run Operation

Add a **Run** or **Analyze** button to initiate the prediction process.

When selected, the application should:

- Validate the input.
- Execute text preprocessing.
- Perform emotion prediction.

- Generate the learning response.
- Display the prediction results.

Step 3: Implement the Clear Operation

Provide a **Clear** button to reset the application.

The operation should:

- Clear the input fields.
- Remove displayed prediction results.
- Reset confidence scores.
- Clear temporary session data while preserving stored history if required.

Step 4: Display Loading Indicators

Use loading indicators (spinners) while processing user requests.

Display appropriate progress messages during:

- Text preprocessing
- Emotion prediction
- Gemini response generation
- Analytics updates

This informs users that the requested operation is in progress.

Step 5: Implement Error Handling

Handle unexpected situations gracefully by displaying informative messages.

Detect and manage errors such as:

- Empty input fields
- Invalid user input
- Model loading failures

- Gemini API communication errors
- Missing files or configuration issues

Provide clear guidance so users can correct the issue and continue using the application.

Step 6: Validate User Interaction

Test the application to verify that:

- Form controls function correctly.
- Run and Clear operations execute successfully.
- Loading indicators appear during processing.
- Error messages are displayed appropriately.
- The application remains stable during normal and exceptional conditions.

Activity 1.4: Visualize Scores and Plotly Charts with Caching

Step 1: Prepare the Analytics Data

Collect the prediction results required for visualization.

The data should include:

- Predicted emotions
- Confidence scores
- Academic fields
- Session history
- Timestamps

Verify that the collected data is complete and free from errors.

Step 2: Generate Interactive Charts

Create Plotly visualizations to present the prediction results.

Typical charts include:

- Emotion distribution chart
- Confidence score chart
- Field-wise emotion analysis
- Prediction timeline

Ensure that the charts are interactive and update dynamically.

Step 3: Apply Caching

Enable caching for frequently used data and chart generation.

Cache:

- Loaded datasets
- Processed analytics data
- Generated visualizations

This reduces repeated processing and improves application responsiveness.

Step 4: Integrate Charts into the Dashboard

Display the generated Plotly charts within the analytics section of the application.

Arrange the visualizations in a clear layout that allows users to easily interpret prediction results and trends.

Step 5: Refresh Visualizations

Automatically update the charts whenever new predictions are generated.

Ensure that the displayed analytics remain synchronized with the latest session history and CSV records.

Step 6: Validate the Visualization Module

Verify that:

- Charts load successfully.

- Interactive features function correctly.
- Cached data reduces loading time.
- Analytics update accurately after each prediction.
- No visualization or performance errors occur.

Milestone 6: User Interaction

Activity 1.1: Validate UI Flow End-to-End

Step 1: Launch the Application

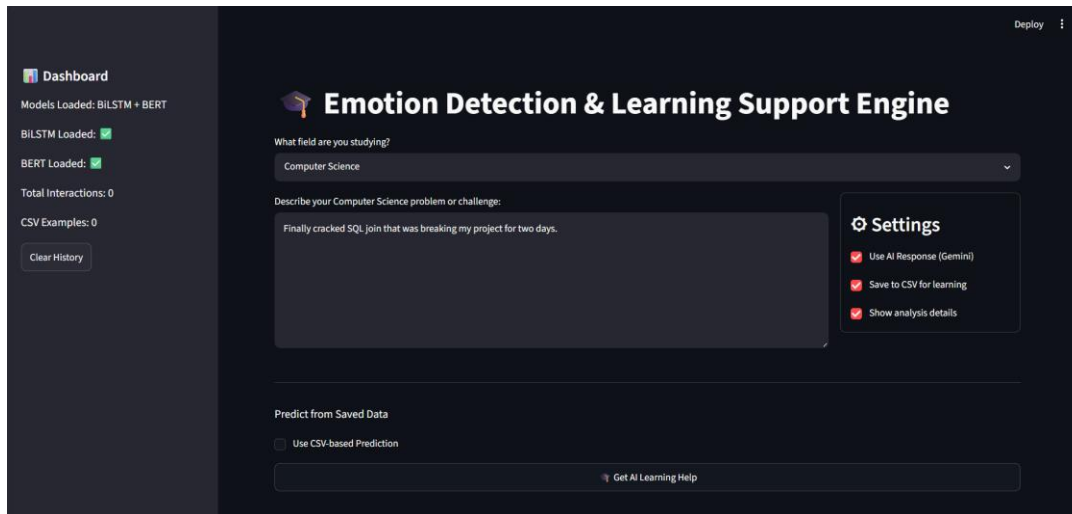
Start the Emotion Detection and Learning Support Engine and verify that the application loads successfully without any startup errors.

Step 2: Verify User Input Flow

Enter a valid academic field and a sample problem description.

Confirm that:

- Input fields accept valid data.
- Required fields are validated.
- Invalid or empty inputs generate appropriate warning messages.

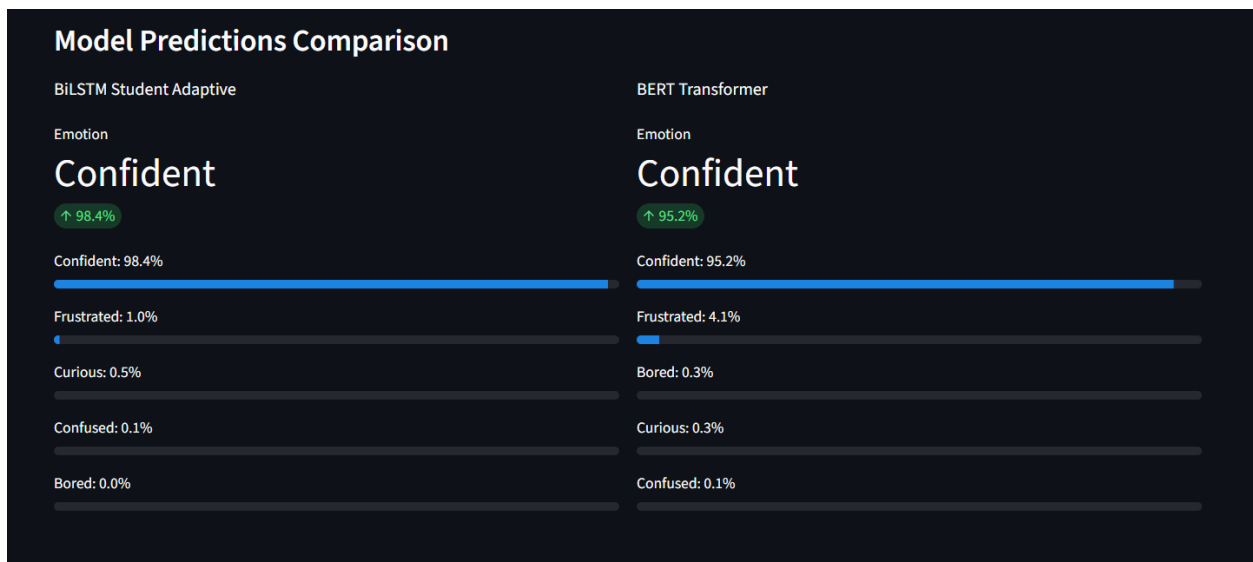


Step 3: Execute Emotion Prediction

Select the **Analyze** button to begin the prediction process.

Verify that:

- Text preprocessing is performed.
- Emotion prediction executes successfully.
- Loading indicators are displayed during processing.



Step 4: Validate Prediction Results

Confirm that the application correctly displays:

- Predicted emotion

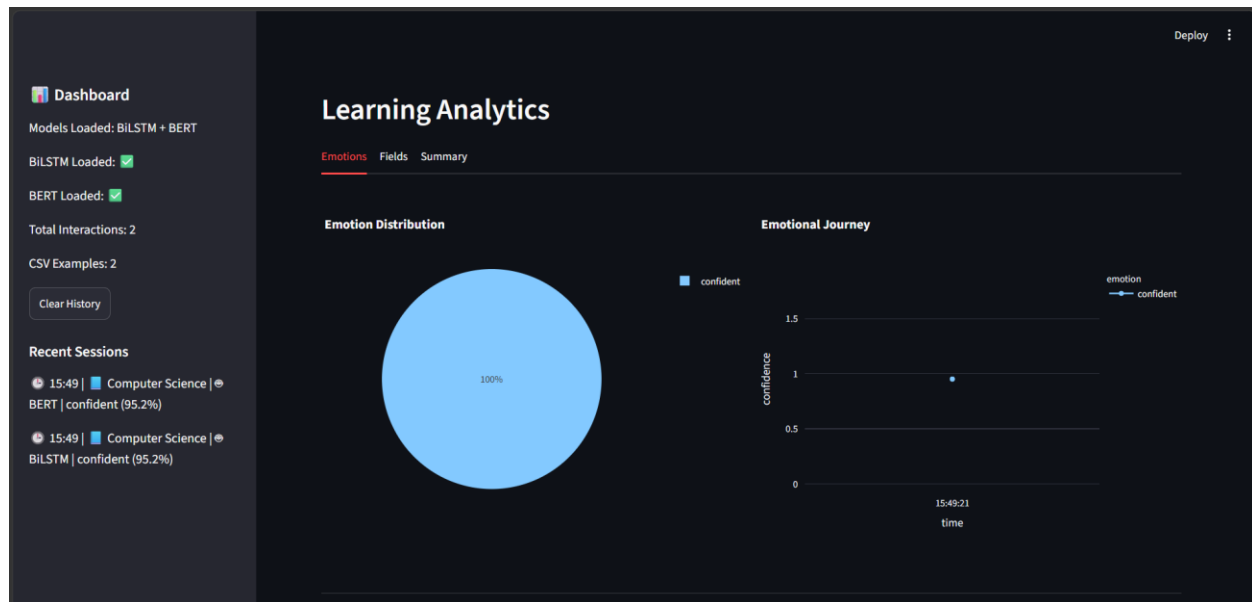
- Confidence score
- Mixed emotions (if detected)
- BiLSTM and BERT model outputs
- AI-generated or template response

Ensure that all displayed results correspond to the submitted input.

Step 5: Verify Analytics and Session Updates

Check that:

- Session history is updated.
- CSV logs are generated.
- Analytics charts refresh automatically.
- Confidence bars and model comparison sections display current results.



Step 6: Test User Interaction Scenarios

Perform multiple test cases using different student inputs.

Verify that:

- The Run and Clear functions operate correctly.

- The AI response toggle updates the displayed response.
- The application remains stable during repeated interactions.
- No unexpected errors occur throughout the workflow.

Activity 1.2: Optimization and Deployment Readiness

- **Cross-Browser Verification**

- Tested application in multiple browsers.
- Confirmed consistent layout, widgets, and visualizations.
- Verified absence of browser-specific UI errors

- **Performance and Caching**

- Confirmed models are loaded only once.
- Verified caching of stored data for faster responses.
- Observed stable response times across repeated interactions.

- **Environment and Final Validation**

- Verify Verified environment variables are correctly configured.
- Confirmed required models and dependencies are available.
- Validated complete end-to-end workflow functionality.
- Confirmed system readiness for deployment.

- **Slide Splitting & PNG Conversion Testing**

Validate that each slide correctly produces:

- one PPTX file
- one PNG preview

Milestone 6: Conclusion

The AI-Powered Learning Assistant represents a significant advancement in how students and educators understand and respond to emotional states during learning. By combining robust text preprocessing, a BiLSTM student-adaptive model, a fine-tuned BERT classifier, and mixed-emotion detection, the system can interpret nuanced learner emotions across five key categories with high accuracy and transparency. On top of this emotion layer, Gemini-driven response generation and carefully designed fallback templates deliver empathetic, field-aware guidance that helps students feel understood while receiving concrete, actionable support.

Built on a modular architecture with reusable Kaggle training notebooks, exported models, and a Streamlit-based interface, the platform offers both technical robustness and strong usability. The dashboard, analytics tabs, and CSV logging enable continuous learning and rich insights into emotional patterns across sessions and subjects, making it valuable for individual learners, tutors, and institutions alike. Looking forward, the assistant provides a strong foundation for extensions such as multilingual emotion detection, voice or multimodal inputs, deeper LMS integration, adaptive learning paths, and instructor dashboards. With its blend of neural models, rule-based enhancements, and human-centered design, the AI Learning Assistant sets a practical benchmark for next-generation emotion-aware educational technology.